
CS 421 --- LL Parsing Activity

Name	Netid

Please write your name/netid legibly in dark ink. Hand in one copy per team. Do not staple or mangle the corners.

Purpose

There is a certain class of grammar for which it is very easy to write a parser without needing any special libraries or tools. Your objectives are to demonstrate how to write such a parser, how to identify a grammar that can use this approach, and how to fix common problems that prevent a grammar from being implemented as a recursive descent parser.

Problem 1 --- Recursive Descent Parsers

Consider these two grammars. Lower case letters will represent literal characters in the input. Upper case letters will represent nonterminal symbols. The character *c* will represent a random character.

Grammar 1

$$\begin{aligned} S &\rightarrow x S y \\ &\quad | E \\ E &\rightarrow a E b \\ &\quad | c \end{aligned}$$

Grammar 2

$$\begin{aligned} S &\rightarrow a S \\ &\quad | E \\ &\quad | F \\ E &\rightarrow b c \\ F &\rightarrow d c \end{aligned}$$

Consider now these HASKELL programs. Assume niceties like deriving `Show`, etc.

```
0 -- Program 1
1 data S = S1 Char S Char
2       | S2 E
3 data E = E1 Char E Char
4       | E2 Char
5 parseS ('x':xs) =
6   let (s,r1) = parseS xs
7       ('y':r2) = r1
8   in (S1 'x' s 'y', r2)
9 parseS xx = parseE xx
10 parseE ('a':xs) =
11   let (e,r1) = parseE xs
12       ('b':r2) = r1
13   in (E1 'a' e 'b', r2)
14 parseE (x:xs) = (E2 x, xs)
```

```
0 -- Program 2
1 data S = S1 Char S
2       | S2 E
3       | S3 F
4 data E = E1 Char Char
5 data F = F1 Char Char
6 parseS ('a':xs) =
7   let (s,r1) = parseS xs
8   in (S1 'a' s, r1)
9 parseS ('b':xs) = parseE ('b':xs)
10 parseS ('d':xs) = parseF ('d':xs)
11 parseE ('b':x:xs) = (E1 'b' x, xs)
12 parseF ('d':x:xs) = (F1 'd' x, xs)
```

Answer the questions on the next page.

Matching Grammar to Code Each of these programs is meant to implement the corresponding grammar. With your team, review the code and be able to explain to each other how it works. Then answer the following questions:

Problem 1) Why do each of the parse functions return a tuple?

Problem 2) How is parsing a non-terminal different than parsing a terminal?

Problem 3) The second grammar has these two line:

```
0 parseE ('b':x:xs) = (E1 'b' x, xs)
```

```
1 parseF ('d':x:xs) = (F1 'd' x, xs)
```

From this example, can you explain how the parseS function determines which of parseE or parseF to call?

Problem 2 --- What Could Possibly Go Wrong?

Grammar 3

$$\begin{array}{l} S \rightarrow S y \\ \quad | x \end{array}$$

Grammar 4

$$\begin{array}{l} S \rightarrow a S \\ \quad | a E \\ E \rightarrow x \end{array}$$

Consider now these HASKELL implementations. We omit the data declarations this time.

```
0 -- Program 3
1 parseS ('x':xs) = (S2 'x', xs)
2 parseS xx =
3   let (s,r1) = parseS xx
4     ('y':r2) = r1
5   in (S1 s 'y', r2)
```

```
0 -- Program 4
1 parseS ('a':xs) =
2   let (s,r1) = parseS xs
3   in (S1 'a' s, r1)
4 parseS ('a':xs) =
5   let (e,r1) = parseE xs
6   in (S1 'a' e, r1)
7 parseE ('x':xs) = (E1 'x', xs)
```

Problem 4) The first program has a problem. What goes wrong? What feature of the grammar causes this problem to occur?

Problem 5) The second program also has a problem. What goes wrong? What feature of the grammar causes this problem to occur?

Problem 3 --- Fixing Left Recursion

Problem 6) Consider these two grammars:

Grammar 5

$$\begin{array}{l} S \rightarrow S a \\ \quad | b \end{array}$$

Grammar 6

$$\begin{array}{l} S \rightarrow b S' \\ S' \rightarrow a S' \\ \quad | \epsilon \end{array}$$

Draw two parse trees for the string baaa, one for each of the above grammars.

Problem 7) Consider these two grammars:

Grammar 7

$$\begin{array}{l} S \rightarrow S a \\ \quad | S b \\ \quad | c \\ \quad | d e \end{array}$$

Grammar 8

$$\begin{array}{l} S \rightarrow c S' \\ \quad | d e S' \\ S' \rightarrow a S' \\ \quad | b S' \\ \quad | \epsilon \end{array}$$

Draw two parse trees for the string deba, one for each of the above grammars.

Problem 8) Describe a conversion procedure to fix a left-recursive grammar.

Given a grammar:
$$\begin{array}{l} S \rightarrow S \alpha \\ \quad | \beta \end{array}$$

Show what the corresponding converted grammar looks like. The α and β here mean ``any arbitrary sequence of terminals and nonterminals.''

Problem 4 --- Fixing Common Prefixes

Problem 9) Consider these two grammars:

Grammar 9

$$\begin{array}{l} S \rightarrow a b \\ \quad | a E \\ E \rightarrow x y \end{array}$$

Grammar 10

$$\begin{array}{l} S \rightarrow a S' \\ S' \rightarrow b \\ \quad | E \\ E \rightarrow x y \end{array}$$

Draw two parse trees for the string axy , one for each of the above grammars.

Problem 10) Consider these two grammars:

Grammar 11

$$\begin{array}{l} S \rightarrow a b \\ \quad | E \\ E \rightarrow x y \\ \quad | a z \end{array}$$

Grammar 12

$$\begin{array}{l} S \rightarrow a S' \\ \quad | x y \\ S' \rightarrow b \\ \quad | z \end{array}$$

Draw two parse trees for the string az , one for each of the above grammars. The second grammar is missing the E production entirely. Why is this necessary?

Problem 11) Describe a conversion procedure to fix a common-prefix rule in a grammar. Given this stylized grammar,

$$\begin{array}{l} S \rightarrow \alpha \beta \\ \quad | \alpha \gamma \\ \quad | \delta \end{array}$$

show what the corresponding converted grammar looks like.

Problem 5 --- Apply It!

Problem 12) This grammar is not LL. Convert it to an equivalent grammar that is LL.

Grammar 13

$$\begin{aligned} S &\rightarrow Sx \\ &\quad | aE \\ E &\rightarrow yay \\ &\quad | yaz \end{aligned}$$

Problem 13) There is a third thing that can go wrong! Look at this grammar and describe what goes wrong. Note, it's not *just* that there is an ϵ production.

Grammar 14

$$\begin{aligned} A &\rightarrow Bc \\ &\quad | x \\ B &\rightarrow c \\ &\quad | \epsilon \end{aligned}$$

Problem 14) Were all these too easy? Try converting this one then.

Grammar 15

$$\begin{aligned} A &\rightarrow Ax \mid By \mid z \\ B &\rightarrow Ai \mid Bj \mid k \end{aligned}$$